

How Invariant is Code Generation to Graph Serialization?

Measuring and Localizing the Fragility

Ram Kedem **Yehonatan Barel** **Ido Azoulay** **Dolev Abudi**
206844144 208646422 212710958 323096099

Abstract

Code generation is among the strongest ways for a language model to answer a graph question, but its invariance to how the graph is written has not been measured per instance on executed code. We test whether it is *invariant*: we perturb one serialization axis at a time, score every instance separately, and execute what the model writes. Across four models and two datasets, generating code raises answer consistency under relabeling and reordering over direct answering in 14 of 16 comparisons, yet no condition in which the model reads the graph is invariant. The fragility is not in transcription, which is correct in over 98% of code records, but in the solving program: duplicated edges are counted twice by three of the four models, and one omitted word leads every model to build the wrong graph on one task, sometimes still scored correct. The sensitivity is also a signal: flagging instances whose answers disagree across permutations catches more than half of the silent errors wherever they vary with the serialization, but almost none of a systematic error, a misread question repeated under every serialization, which is the residual no serialization intervention here removes.

1 Introduction

Large language models (LLMs) typically receive graphs through serialization: the conversion of nodes and edges into a sequential text representation, such as a list of edges. Renaming nodes, reordering edges, or changing the notation preserves graph structure and query semantics while changing the token sequence presented to the model, and prior work shows that such changes can affect reasoning performance and lead to different answers (Fatemi et al., 2024; Herbst et al., 2026).

Generating and executing code offers another way to solve these problems. Code-based approaches perform strongly on graph reasoning tasks (Cai et al., 2024; Finkelshtein et al., 2026),

but higher accuracy does not necessarily imply greater invariance across serializations (Herbst et al., 2026).

We ask: *How robust are LLM-generated code solutions across different serializations of the same graph, and what observable failures accompany a loss of robustness?* We compare direct answering with two code modes, one that transcribes the serialized graph into program data and one that receives the data at execution. Per instance we score both correctness and whether the answer survives the perturbation, since equal accuracy can hide mistakes on different questions, and we inspect the programs for transcription, execution and logic errors. Measurement establishes that the problem is real; localization says where it lies, and so which remedies can apply. Our contributions are (i) a per-instance, single-axis invariance measurement of *executed* code over four models and two datasets; (ii) a localization of the failures in the solving program rather than in the copied graph; (iii) a graph-type omission ablation showing that a model infers an omitted graph property from incidental nearby wording and that the resulting wrong graph can still be scored correct; and (iv) a mitigation study on all four models in which the sensitivity becomes an error detector, together with the error class it cannot see.

2 Related Work

Graph serialization and reasoning: Fatemi et al. (2024) introduce GraphQA, a benchmark of graph reasoning tasks, and compare graph encodings and prompting strategies. Their results show that the textual representation of a graph affects LLM performance. Herbst et al. (2026) study invariance more directly, decomposing serialization into node labeling, edge encoding (including structure and order), and syntax. We build on this decomposition and extend the comparison to LLM-generated code solutions.

Executing generated code: CodeGraph (Cai et al., 2024) generates Python for graph questions under several encodings and so already gives evidence on encoding sensitivity in code-based reasoning; its per-encoding accuracy comparisons motivate our complementary focus on matched instances and on inspecting the programs. Our template is zero-shot, where CodeGraph supplies a worked example.

Finkelshtein et al. (2026) compare prompting, tool use and code generation and find code generation most effective; their Graph-as-Code mode computes iteratively over a preloaded table, and our injected-data condition adapts the idea to one program per distinct prompt, run on data injected at execution, rather than reproducing that protocol.

3 Methodology

3.1 Instances and Data

An instance is a graph G , a question q and a reference answer y ; every variant preserves both graph and query semantics. When labels change we rename the query consistently and map node-valued answers back before scoring. The reference is a fixed default serialization, not a canonical labeling shared by isomorphic graphs.

GraphQA: We generate undirected, unweighted graphs of 5–20 nodes from the seven families of Fatemi et al. (2024) (Erdős–Rényi, Barabási–Albert, scale-free, stochastic block, star, path, complete). Tasks cover edge existence, node degree, node and edge counts, neighbour and non-neighbour listing and cycle detection; questions are sampled per graph and answers computed with NetworkX. This is a GraphQA-style sample, not the original paper’s evaluation instances.

Erdős: Selected test tasks from Guo et al. (2025) add harder problems: shortest paths, triangle counting, connected components, diameter, density and bridges. The loader keeps the original task wording and queries, recovers graph data from the prompts and checks it against the dataset’s edge field. Path predictions are additionally validated for validity and optimal cost.

3.2 Serialization Perturbations

The reference uses an edge list: GraphQA’s natural language “adjacency” encoding and Erdős’s native graph wording. Following Herbst et al. (2026), we vary four factors: **relabeling** permutes node identifiers; **order** changes the presentation sequence;

structure switches between edge lists and adjacency lists or presents undirected edges in both directions; and **syntax** changes surface notation, including plain text, JSON, and NetworkX code. Each variant changes one factor only, so a difference in answers can be attributed to that factor.

Variants are built from structured graph data with fixed seeds; the generator asserts that each changes only its designated factor and drops no-ops. Relabeling uses a sorted identity reference and sorts again after renaming, so it holds the ordering rule fixed rather than individual positions, and can therefore still move edges. An instance has up to 13 forms (12 on Erdős): the reference, the sorted identity and three random relabelings, up to three orderings, up to two structures and up to three syntaxes. M3 runs on all of them, but only relabeling and order change what it receives.

3.3 Interaction Modes

We use three single-turn, zero-shot modes; none includes a solved example. The modes form a ladder in which each step removes one task from the model: M1 reads the graph, holds it, and computes; M2 delegates the computation to a program; M3 additionally removes the copying of the graph. Figure 1 shows what each mode receives and what is scored.

Direct answering (M1): Graph text and a question, with the final answer in a boxed expression. Nothing is executed.

Code from text (M2): The same graph text as M1, and a Python template to fill: the model declares nodes and edges first, then assigns ans. We execute the code between the markers and score the value of ans, not printed output. The declared lists make transcription inspectable in either library arm.

Graph-as-Code (M3): The task and question only, with instructions to use existing nodes and edges variables whose values it never sees. The harness supplies those lists in the variant’s labels and order at execution, so M3 removes copying while retaining construction and computation.

Comparing M2 and M3: M3 also removes the model’s ability to inspect the particular graph, so an M2–M3 difference is diagnostic, not a causal estimate of transcription cost; transcription itself is measured inside M2, from the declared lists.

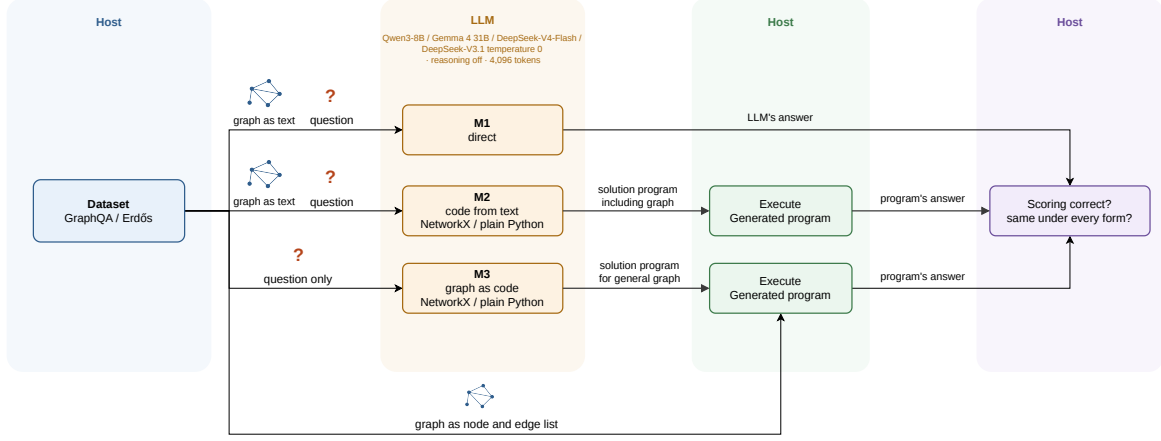


Figure 1: One instance through the three interaction modes and the shared scorer.

Both code modes are crossed with a library arm: NetworkX permitted, or the standard library only, which forces the algorithm to be written out. With M1 this gives five conditions. The arms are never pooled, because they offer different tools and different opportunities to recover intermediate graph objects.

3.4 Failure Analysis

We separate execution failures (including timeouts), unparseable answers and programs that assign a literal answer without computing. Remaining wrong answers are labelled *transcription* when the declared graph differs from the input, *construction* when a recovered NetworkX graph differs from that declaration, and otherwise *logic*; programs with too little recoverable structure stay *unverifiable*. Correct answers are checked for graph mismatches separately, since a corrupted graph can leave the queried property unchanged.

These labels are diagnostic, not a causal trace: declarations are recovered after execution, the NetworkX object is chosen by edge overlap, edge comparisons ignore weights and multiplicity, and in plain Python a construction error falls into *logic* because no graph object exists, so we read them per library arm.

4 Experimental Setup

4.1 Models

We evaluate four instruction-tuned models spanning nearly two orders of magnitude in parameter count and three developers: Qwen3-8B (Alibaba, 8B), Gemma 4 31B (Google, 31B), and the mixture-of-experts DeepSeek-V4-Flash (284B, 13B active) and DeepSeek-V3.1 (671B, 37B active).

Inference and execution: All models are served through one gateway under identical settings: temperature zero, a 4,096-token limit and, where the provider exposes the switch, reasoning disabled. Prompts are held fixed across variants except for the designated graph/query change. Responses are cached by model and prompt text; programs run in separate processes with a wall-clock timeout. Identical M3 prompts reuse one generated program across many injected graphs: those are distinct evaluations but not independent generations, though relabeling and order still affect the program’s behaviour at execution.

Scale: Every instance is evaluated in all five conditions under every retained form: 44,625 evaluations per model on GraphQA and 71,230 on Erdős. Caching and repeated M3 prompts leave 56–57% of them as distinct requests.

4.2 Baselines and Metrics

M1 is the direct-answer baseline, while M3 provides a comparison without graph transcription. We report answer accuracy separately from program-compliance diagnostics: a literal answer can be correct while labeled as lacking computation. Node-valued outputs are remapped to the original labels, and unordered answers are normalized before comparison. A floating-point prediction $\hat{y}$ is correct when $|\hat{y} - y| \leq \max(0.01, 0.01|y|)$; consistency instead compares values rounded to four decimal places.

For instance i and axis a , let $V_{i,a}$ hold the reference and its retained variants and $k_{i,v}$ a normalized answer key. Then $I_{i,a} = 1$ iff every answer in $V_{i,a}$ parses and $|\{k_{i,v}\}| = 1$; groups need a reference

Model	Direct		Native		NetworkX	
	<i>I</i>	<i>R</i>	<i>I</i>	<i>R</i>	<i>I</i>	<i>R</i>
<i>GraphQA</i>						
Qwen3-8B	76	70	94	94	88	85
Gemma 4 31B	97	87	96	95	95	95
DeepSeek-V4-F	90	76	93	75	92	79
DeepSeek-V3.1	89	74	97	85	97	84
<i>Erdős</i>						
Qwen3-8B	50	51	86	89	62	65
Gemma 4 31B	92	96	95	100	96	100
DeepSeek-V4-F	76	80	92	96	95	99
DeepSeek-V3.1	84	88	94	98	92	97

Table 1: Consistency *I* and all-forms correctness *R* (%), averaged over relabeling and edge order.

and at least one other distinct variant. We report its mean as *I*, and the fraction correct on every form as *R*. Consistency is not correctness: a stable wrong answer scores *I* = 1, and two equally valid shortest paths lower *I* without lowering *R*, so the two are always read together. Comparisons across axes must account for the differing number of forms per instance.

Paired intervals for code-versus-direct comparisons resample graph clusters jointly across conditions, weighting tasks and axes equally; the resulting 95% percentile intervals describe graph-sampling uncertainty conditional on the collected responses. Variability under repeated identical prompts is measured separately, and invariance is reported beside it rather than corrected by it: temperature zero is not treated as a determinism guarantee.

5 Results and Discussion

5.1 Accuracy and Invariance

Table 1 reports *I* and *R* averaged over the two permutation axes. Over the 16 code-versus-direct comparisons (four models, two datasets, two library arms), code raises *I* in 14 and *R* in 15. The exceptions are on GraphQA: Gemma, whose direct answers are already the most consistent (97 → 96 and 95; the NetworkX drop, −1.9 points [−3.2, −0.6], is significant), and DeepSeek-V4-Flash’s plain-Python *R* (76 → 75, within noise). The gains are largest where direct answering is weakest: Qwen3-8B on Erdős moves from *I* = 50 to *I* = 86, and its canonical accuracy from 71% to 95%.

No condition in which the model reads the graph is invariant: the best cell is 97 (M3, which receives no graph text, reaches 100 in four of eight GraphQA arms). The two metrics must be read

together in both directions. DeepSeek-V3.1 on GraphQA has *I* = 97 but *R* = 85: it answers a different question than the benchmark intends on one task, identically under every serialization. Its Erdős shortest-path cell is the converse, *I* = 47 with *R* = 99, because different shortest paths are equally correct.

Re-asking an identical prompt four times on about 100 stratified instances bounds how much of the flipping is the perturbation. Every Qwen3-8B cell exceeds its own floor; in its Erdős plain-Python arm the floor is 0–1% against flip rates of 8–14%, so essentially all of that instability is the perturbation. For the other three models the excess is +4 to +10 points where its interval excludes zero, mostly on relabeling, and within noise elsewhere at *n* = 100: underpowered rather than null.

Presentation interacts with the condition. On GraphQA, Qwen3-8B’s plain-Python consistency is 93–96 on the permutation axes but 60 on structure, and its NetworkX consistency falls to 61 on syntax, both below direct answering (68, 73); on structure and syntax code trails direct answering in 5 of 32 comparisons. Gemma stays at 94 or above on every axis in both arms.

5.2 Failure Localization

Transcription is not the dominant failure. The graph the program *declares* matches the input in 98.1–99.9% of ordinary code records. It is, however, not self-validating: among records whose declaration is wrong, 64% (GraphQA) and 60% (Erdős) still produce the correct answer, so accuracy cannot certify copying.

The failures sit in the solving code. When undirected edges are listed in both directions, plain-Python degree code counts each edge twice in three of the four models: accuracy falls from 99 to 5 (Qwen), 100 to 19 (DeepSeek-V3.1) and 100 to 65 (DeepSeek-V4-Flash) out of 100, while Gemma is unaffected and, with NetworkX permitted, a Graph merges the duplicates (97–100). Permitting NetworkX costs Qwen 22 points on Erdős (69% against 91%), 90.7% of its wrong NetworkX records being execution failures, and the other three models at most one.

Finally, serialization includes what is *not* written. Our injected-data control omitted the word “undirected” from the Erdős prompt. Where the preamble asserts the graph is undirected, no program built a directed graph, and where it says nothing only one did; where it names “directed” only to define

Data	Model	Flag	Caught	Abst. (cov.)
GraphQA	Qwen3-8B	9.1	88.9	99.7 (90.9)
GraphQA	Gemma 4 31B	5.1	75.0	99.4 (94.9)
GraphQA	DeepSeek-V4-F	9.1	26.3	82.4 (90.9)
GraphQA	DeepSeek-V3.1	3.4	1.1	86.8 (96.6)
Erdős	Qwen3-8B	17.1	72.7	98.5 (82.9)
Erdős	DeepSeek-V4-F	9.5	53.8	99.4 (90.5)
Erdős	DeepSeek-V3.1	8.1	1 of 1	100 (91.9)
Erdős	Gemma 4 31B	5.0	—	100 (95.0)

Table 2: Permutation disagreement as an error detector, plain-Python arm (%). Caught: wrong answers that returned one (a crash needs no detector). Abst.: accuracy when answering only if all permutations agree (share answered).

a conditional that does not apply, *all four models* built a directed graph on every undirected instance they could be observed on, and every arm scored 28/100. Stating the type gives 100/100. The error is also partly invisible: 12 of the 84 undirected answers were still correct, and where the preamble describes both cases Gemma built a directed graph for 69% of recovered graphs, nearly three quarters still correct.

5.3 Ablations and Mitigation

Because the fault is in the program, not the transcription, we test levers at inference time. The first uses the sensitivity itself: flag an instance when its answers disagree across permutations (Table 2).

Where errors vary with the serialization it catches most of them: 73–94% of silent errors for Qwen and Gemma on GraphQA (both arms) and Qwen’s Erdős plain Python at 5–17% flag rates, where abstaining answers 83–95% of instances at 97.3–99.8% accuracy, and 54–56% on Erdős for DeepSeek-V4-Flash’s plain Python and Qwen’s crash-heavy NetworkX arm. For Qwen’s GraphQA plain Python one extra relabeled call is enough: 15 of 18 errors at a 4.4% flag rate.

The two cells where it fails are the result. Both DeepSeek models misread `disconnected_nodes` identically under every serialization, and it holds 59–99% of their GraphQA errors, none caught; Gemma makes the same misreading on only 16 graphs and 12 are caught. *The blind spot is a property of the error, not of the model.* Detection finds unstable errors, not wrong beliefs.

Voting over the permuted answers moves code-arm accuracy by -2.4 to $+6.4$ points, and voting over the permuted programs, each run on one fixed graph, by -2.3 to $+5.3$ (on 300 instances per dataset): both help where errors are unstable and amplify a majority misreading where they are

not. An exact canonical labeling renders every relabeling and reordering of every instance to one prompt; on a fixed-stride subset covering every task (400 GraphQA and 150 Erdős instances) it is significantly more accurate than random labels in 8 of 24 cells and less in one, on the ambiguous task. One solver per task, with no graph in the prompt, is free on Erdős for the three stronger models once told the graph type, but a single program decides a whole task: one misread task costs each of them 3–11 points in a GraphQA arm, and nonexistent NetworkX routines cost Qwen 4–7. A fixed adapter reconciles the prompt’s two answer conventions, dropping a solver’s own top-level call and returning an assigned but unreturned ans; it changes 17 Qwen programs and no others (Qwen’s GraphQA plain-Python solver: 72% without it). A one-sentence hint to count duplicate edges once, run later on two models against the original run on another machine (so provider drift is not excluded), coincides with duplicated-edge degree of 100 for DeepSeek-V3.1 (from 34 and 19) but 87 for Qwen, whose canonical degree falls from 99 to 86 and consistency on three unrelated axes by 6–13 points.

6 Conclusion and Limitations

Executed code is more robust than direct answering to relabeling and edge order for most models, though not on every axis, and it is not invariant; the fragility does not enter where expected: graphs are transcribed almost perfectly, and the failures sit in the solving code. What remains is partly usable, since permutation disagreement detects unstable errors at several calls per instance and abstaining on it is nearly always right where they occur, and partly beyond these levers: a model that misreads the question repeats that mistake under every serialization.

We are limited to four models on one gateway, small graphs (5–20 and 4–34 nodes) and zero-shot prompts where NetworkX is permitted, not required; every experiment but the repeat baseline uses one generation per prompt, the deduplication hint covers two models, our pre-registered detection and per-task-solver targets are not met, and the canonical-labeling one only for two models. Where a question names no node, one shared M3 program can decide a task.

References

- Qiaolong Cai, Zhaowei Wang, Shizhe Diao, James Kwok, and Yangqiu Song. 2024. CodeGraph: Enhancing graph reasoning of LLMs with code. *Preprint*, arXiv:2408.13863.
- Bahare Fatemi, Jonathan Halcrow, and Bryan Perozzi. 2024. Talk like a graph: Encoding graphs for large language models. In *The Twelfth International Conference on Learning Representations*.
- Ben Finkelshtein, Silviu Cucerzan, Sujay Kumar Jauhar, and Ryen White. 2026. Actions speak louder than prompts: A large-scale study of LLMs for graph inference. In *The Fourteenth International Conference on Learning Representations*.
- Xiaojun Guo, Ang Li, Yifei Wang, Stefanie Jegelka, and Yisen Wang. 2025. G1: Teaching LLMs to reason on graphs with reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 38.
- Daniel Herbst, Lea Karbevskaya, Divyanshu Kumar, Akanksha Ahuja, Fatemeh Gholamzadeh Nasrabadi, and Fabrizio Frasca. 2026. Lost in serialization: Invariance and generalization of LLM graph reasoners. In *ICML Workshop on Graph Foundation Models*.